

<https://github.com/X57FI8W9S/kbot-box-top-locomotion>

https://github.com/X57FI8W9S/kbot-box-top-locomotion/blob/main/FINAL_REPORT.md codex did a 99% full auto version of this project, i'm going to tell it to go for a second version. but when i asked if over 40 reward parameters is too much, he said it's normal because we're using rewards as diagnostics, so probably on this version i'm going to tell it to do a dedicated diagnostic module with a dashboard, and maybe have another agent review the training results and go 99,99% "full" auto. robot walks, but has a bad walking posture, and if you change the rewards for better posture, yaw goes bad. please check the project and final report, what would be good diagnostics indicators? i at least want step length, step duration, ratio of double to single ground contact support L and R, ratio of full foot sole support to step duration (duration is heel landing to opposite heel landing), rolling averages window probably should be adjusted to 5 full cycles. let's first think what indicators you would need to approve or reject, or compare a new checkpoint to an old checkpoint and to choose when to stop a training run.

Yes: **make a dedicated diagnostics/evaluator module**, and keep it separate from the reward function. Over 40 reward terms is not automatically wrong, but in your report the problem is clearly that many terms became **overlapping control pressures**, not just diagnostics. The report itself says the remake should use fewer reward terms, avoid duplicate constraints, add better foot contact observability, and select checkpoints with video + rolling metrics, not scalar reward alone. ([GitHub](#))

The project is a simplified flat-ground, forward-walking PPO task for a top-heavy KBot-like biped, so the diagnostic dashboard should answer one question: **is this checkpoint walking in a physically acceptable way, or did it find another exploit?** The report already notes that awkward behavior can come from the simplified body, limited actuation, reward design, and contact modeling. ([GitHub](#))

Main recommendation

For v2, I would split things into three layers:

1. **Reward terms**: only what trains the policy.
2. **Diagnostic metrics**: many metrics, no direct optimization pressure.
3. **Evaluator/scorecard**: approves, rejects, ranks, or stops checkpoints.

So yes, use "rewards as diagnostics" only in the sense of logging components, but do **not** let all diagnostic ideas become reward weights. Your final report already shows this risk: it had 43 reward terms, including many foot-lane, frontal-plane, hip, torso, flatness, yaw, and anti-collapse terms. ([GitHub](#))

Metrics I would absolutely add

Your proposed metrics are exactly the right direction. I would group them like this.

1. Basic pass/fail health

These are hard gates before comparing elegance:

Metric	Why it matters
Timeout/survival fraction	Rejects falling policies.
Termination cause counts	Tells whether it falls, body-contacts, crouches, locks knees, etc.
Undesired body contact time	Detects dragging/collapsing.
Root height mean/min/p5	Detects crouch-walk.
Knee min/max/mean	Detects locked-knee or over-crouched gait.
Both-feet-airborne fraction	For walking, should be near zero unless you intentionally train running.

The current report already treats episode length, timeout-only behavior, and termination penalty as stability signals. ([GitHub](#))

2. Command tracking and straightness

These should be required for “walking forward”:

Metric	Use
<code>vx_mean</code> , <code>vx_error_mean</code> , <code>vx_error_p95</code>	Does it follow commanded speed?
<code>vy_mean_abs</code> / lateral drift per meter	Detects sideways walking.
<code>yaw_rate_mean_abs</code> , <code>yaw_rate_p95</code> , <code>yaw_drift_per_meter</code>	Detects yaw going bad when posture improves.
Heading error mean/p95	Detects walking at an angle while reward still looks good.
Path curvature	Better than yaw alone: “how much does the path curve over distance?”

For your specific issue, **yaw should be a hard constraint, not just one score among many**. If a new checkpoint improves posture but yaw drift is much worse, it should be marked “not better,” not “maybe better.”

3. Posture and balance quality

The report already discovered the key distinction: **bias is not the same as oscillation**. **torso avg** tells lean/bias, while **torso rms** tells movement/oscillation; high RMS with near-zero average is a different failure mode from persistent lean. ([GitHub](#))

Add:

Metric	Use
Torso/root roll mean	Persistent lateral lean.
Torso/root roll RMS	Oscillation/wobble.
Torso/root pitch mean/RMS	Forward/backward posture.
Root yaw drift	Orientation stability.
Hip roll/yaw mean and RMS, L/R separated	Detects compensating hip posture.
Pelvis/root lateral acceleration RMS	Balance jerk/wobble.
COM projection margin over support polygon	Best balance metric if feasible.
Capture-point or extrapolated-COM margin	Useful later for push robustness.

The report’s late checkpoints show why this matters: one checkpoint had almost zero torso signed bias but remaining oscillation, so side-view/video became the deciding factor. ([GitHub](#))

4. Gait event metrics

This should be the core of the new diagnostic module.

Define contact events from heel/toe/sole contact if available. If not available, start with foot contact sensor rising edges, then improve the asset with heel/toe contact bodies or sensors.

For each detected step:

Metric	Definition

Left heel strike / right heel strike timestamps	Contact rising edge for each foot.
Step duration	For L step: $t_{R_landing} - t_{L_landing}$; for R step: $t_{L_next_landing} - t_{R_landing}$.
Full gait cycle duration	L landing to next L landing, or R landing to next R landing.
Step length	Forward projected distance between previous stance-foot landing point and opposite-foot landing point.
Step width	Lateral projected distance between feet at landing.
Cadence	$steps / second$.
Duty factor L/R	Contact time of foot divided by cycle or step duration.
Swing duration L/R	Non-contact time per foot.
Stance duration L/R	Contact time per foot.
Double-support fraction	Time both feet contact / step or cycle duration.
Single-support L/R fraction	Time only L or only R contacts / step or cycle duration.
Airborne fraction	Time neither foot contacts. For walking, should be close to zero.

Your proposed rolling window should be **event-based**, not just seconds-based: use **5 full gait cycles**, e.g. 5 left-to-left cycles, which equals about 10 steps. That avoids changing the smoothing window just because cadence changes.

5. Sole/contact quality

This is important because your report says the current code does **not** have a true sole contact area reward; `stance_foot_flat_12` is only an orientation proxy. ([GitHub](#))

So for v2, ask Codex to add real contact diagnostics:

Metric	Good implementation
Full sole support ratio	Time heel + toe/forefoot contact are both active during stance.
Heel-only ratio	Detects walking on heel edge.

Toe-only ratio	Detects tiptoe exploit.
Lateral-edge contact ratio	Detects walking on side of foot.
Contact force distribution heel/toe	Detects fake flat foot with all load on one point.
Stance foot pitch/roll	Orientation proxy, still useful.
Foot slip during stance	Horizontal foot velocity while contact force is active.
Landing impulse	Detects stomping.

Your proposed “ratio of full foot sole support to step duration” should be:

`full_sole_support_time_for_stance_foot / step_duration`

and also:

`full_sole_support_time_for_stance_foot / stance_duration`

The second one is cleaner, because it ignores swing time.

6. Symmetry metrics

These are critical for “good posture but yaw bad” cases.

Metric	Use
L/R step length difference	Detects asymmetric stride.
L/R step duration difference	Detects limp or phase imbalance.
L/R stance duty factor difference	Detects one foot doing all the work.
L/R full-sole support difference	Detects one bad contact side.
L/R hip roll/yaw bias difference	Detects posture compensation.
L/R knee range difference	Detects asymmetrical leg strategy.
L/R foot yaw/toe-in difference	Detects turning bias.

A checkpoint should not be accepted just because averages look good. It should pass **left-right symmetry**.

7. Smoothness, effort, and hardware sanity

These should not dominate selection, but they catch ugly policies:

Metric	Use
Action rate RMS/p95	Twitchy commands.
Joint velocity RMS/p95	Mechanical wobble.
Joint acceleration RMS/p95	Violent motion.
Torque RMS/p95/max	Motor stress.
Mechanical power estimate	Energy usage.
Cost of transport proxy	Compare efficient vs wasteful gait.
Joint limit proximity	Detects risky posture.

The report already had action-rate, torque, acceleration, and hip/yaw wobble terms; keep them as diagnostics even when not used strongly as rewards. ([GitHub](#))

Approval / rejection logic

I would not use one giant score at first. Use **hard gates + comparison score**.

Hard reject a checkpoint if any of these happen

Reject even if scalar reward is high:

Failure	Example
Fall/body contact	Any non-foot body contact in clean eval.
Timeout fraction low	Cannot complete 30s/60s evaluation.
Yaw drift too high	Posture improves but robot turns.
Lateral drift too high	Robot walks diagonally/sideways.
Airborne fraction high	It is hopping when task is walking.
Toe-only/edge-contact dominates	Looks like walking but contact is fake.
Step detection fails	No consistent L/R gait cycle.

Severe L/R asymmetry

One leg is dragging or doing all support.

Compare old vs new checkpoint with a scorecard

Use the current best checkpoint as baseline, not scalar reward. The report gives good late reference numbers: `model_11791.pt` had speed mean `0.39986`, command mean `0.41960`, yaw-rate mean `0.00568`, torso avg `-0.00087`, torso RMS `0.01888`, and hip roll/yaw RMS `0.08566`. ([GitHub](#))

A new checkpoint is “better” only if:

1. It passes all hard gates.
2. It does not regress yaw/straightness beyond tolerance.
3. It improves the target weakness, e.g. sole contact or posture.
4. It keeps step timing and L/R symmetry sane.
5. Video does not contradict metrics.

Example rule:

```
approve_new_checkpoint =  
  pass_hard_gates  
  and yaw_drift_per_meter <= old * 1.10  
  and lateral_drift_per_meter <= old * 1.10  
  and survival_rate >= old  
  and target_metric_improves >= 15%  
  and no_visual/contact_regression
```

So if you ask for “better posture” and yaw gets worse, the evaluator should say:

REJECT: posture improved, but yaw/heading regression violates hard gate.

That is exactly the tradeoff you described.

Stop/continue training rules

Stop and save best checkpoint when

Stop the run if:

- The evaluator score plateaus for several checkpoints.
- Scalar reward keeps rising but diagnostic score does not.
- The target weakness is solved but other hard metrics start regressing.

- The latest checkpoint is not better than an earlier checkpoint under the fixed eval suite.
- Video and metrics agree that gait quality is acceptable.

The report explicitly says not to use scalar reward alone because visually worse gaits can score better. ([GitHub](#))

Kill or branch a run early when

Stop or branch if:

- Survival/timeout does not improve after the bootstrap phase.
- Yaw drift gets worse while posture terms improve.
- Full-sole support gets worse while foot-flatness reward improves.
- Step events become irregular or one foot dominates support.
- The policy discovers crouch, tiptoe, locked-knee, sliding, or diagonal-walk exploits.

What I would tell Codex to build

Use something like this:

Add a dedicated diagnostics package for checkpoint evaluation.

Goal:

Do not add these metrics as reward terms by default. Build a separate evaluator that runs fixed rollouts, extracts gait/contact/posture metrics, produces CSV/JSON/plots/dashboard, and compares checkpoints using hard gates plus a scorecard.

Required metrics:

- Survival, timeout fraction, termination causes, body contact time.
- vx tracking, vy drift, yaw-rate, heading error, yaw drift per meter, path curvature.
- torso/root roll and pitch mean/RMS/p95, separated into bias and oscillation.
- hip roll/yaw mean/RMS, left/right separated.
- root height, knee min/max/mean, joint limit proximity.
- left/right touchdown timestamps, step duration, cycle duration, cadence.
- step length, step width, foot clearance, swing duration, stance duration.
- double-support fraction, single-support L/R fraction, airborne fraction.
- full sole support ratio per step and per stance duration.
- heel-only, toe-only, edge-contact ratios if contact points/sensors exist.
- stance foot slip, landing impulse, contact force distribution.
- action rate, joint velocity, joint acceleration, torque, power/CoT proxy.
- L/R symmetry metrics for step length, duration, duty factor, sole support, joint ranges.

Rolling window:

Use 5 complete gait cycles, not a fixed time window. For L/R step metrics, this usually means the last 10 steps.

Outputs:

- diagnostics/<checkpoint>/metrics.json
- diagnostics/<checkpoint>/gait_cycles.csv
- diagnostics/<checkpoint>/step_events.csv
- diagnostics/<checkpoint>/summary.md
- diagnostics/<checkpoint>/dashboard.html
- plots for speed/yaw, torso bias/RMS, hip bias/RMS, contact states, step length/duration, support ratios, sole contact quality, symmetry.

Checkpoint comparison:

Implement `compare_checkpoints.py` old.pt new.pt that returns:

APPROVE, REJECT, or REVIEW_VIDEO.

Use hard gates for falls, yaw regression, lateral drift, contact degeneracy, missing gait cycles, and severe asymmetry.

Never rank by scalar reward alone.

My strongest suggestion: **make yaw/heading, contact quality, and L/R gait symmetry hard gates**, not soft dashboard decorations. Otherwise the next agent will keep finding “better” reward configurations that improve one visible posture problem while silently breaking straight walking.

probably forgot to tell you, later policies were ok on yaw, one was good, but they never stopped tiptoeing and had a persistent hip roll, it tilted left (hip/torso/boxtop is supposed, or it is hard to avoid, to oscillate around roll axis because we don't have a joint between hip and upper body, but the oscillation should be around the horizontal hip position, no persistent roll on average). also a bit slow. persistent differences between rolling average l and r joint positions. knees are probably too bent and steps way too short. i'll just print your response and give it to codex.

Yes, that changes the evaluator priorities. The main failure mode is no longer yaw; it is:

stable but ugly gait: tiptoe walking, persistent left roll bias, asymmetric L/R joint averages, crouched knees, short steps, and under-speed.

Here is a clean addendum you can give Codex.

Addendum for v2 diagnostics: current failure modes

Later policies are acceptable on yaw, but still fail on gait quality. The evaluator should not only check “does it walk straight?” It must reject policies that walk by exploiting bad posture or bad contacts.

Current observed problems:

1. **Tiptoe walking / poor sole contact**
 - Feet do not settle onto the full sole.
 - Heel support appears weak or absent.
 - Foot-flat orientation rewards are not enough; need real contact diagnostics.
2. **Persistent hip/torso/box roll bias**
 - Some roll oscillation is expected because the robot has no joint between hip and upper body.
 - But the oscillation should be centered around neutral/horizontal.
 - A nonzero mean roll is bad even if the RMS is acceptable.
 - Track both:
 - `roll_mean` = persistent lean/bias
 - `roll_rms_centered` = oscillation around the mean
3. **Persistent L/R joint average differences**
 - The rolling average of left and right joints should be symmetric after applying each joint's sign convention.
 - Do not compare raw L/R values blindly, because some mirrored joints naturally have opposite signs.
 - Add normalized symmetry metrics.
4. **Knees probably too bent**
 - The robot walks in a crouched posture.
 - Need knee angle mean/min/max and root height diagnostics.
 - Reject policies that achieve walking by staying excessively crouched.
5. **Steps too short**
 - Robot moves forward but with tiny steps and high posture compensation.

- Need step length, cadence, step duration, and stride length.
- Compare actual step length to expected step length from commanded velocity and step duration.

6. **Slightly too slow**

- Current example is about 0.45 m/s for command 0.52 m/s.
 - That is not catastrophic, but checkpoint ranking should prefer closer speed tracking once gait/posture pass.
-

Updated checkpoint evaluator priorities

Use this order:

1. Hard safety gates

Reject if:

falls
non-foot body contact
timeout fraction too low
root height too low
knees too crouched
uncontrolled yaw/lateral drift
no consistent alternating L/R steps

2. Contact-quality gates

Reject if:

toe-only contact ratio too high
heel contact ratio too low
full-sole contact ratio too low
stance foot slip too high
landing/contact impulses too violent
left/right sole contact quality differs too much

Required contact metrics:

heel_contact_L/R

toe_contact_L/R
forefoot_contact_L/R
full_sole_contact_L/R
heel_only_ratio_L/R
toe_only_ratio_L/R
full_sole_ratio_per_stance_L/R
full_sole_ratio_per_step_L/R
stance_slip_L/R
contact_force_heel_toe_distribution_L/R

If true heel/toe contact bodies are not available, add them. Do not rely only on foot pitch/roll orientation.

3. Roll-bias gates

The robot may oscillate in roll, but it should not lean left or right on average.

Track:

base_roll_mean
base_roll_rms_centered
torso_or_box_roll_mean
torso_or_box_roll_rms_centered
hip_roll_mean_L/R
hip_roll_rms_L/R
support_center_offset_mean
COM_lateral_offset_over_support_mean

Reject if:

abs(base_roll_mean) too high
abs(box_roll_mean) too high
abs(hip_roll_bias) too high
roll oscillation is centered around a tilted posture

Important distinction:

bad: mean roll = -0.08 rad, RMS = 0.03 rad
okay: mean roll $\approx$ 0.00 rad, RMS = 0.03 rad

The second one oscillates around neutral. The first one is persistently tilted.

4. L/R symmetry gates

Compute symmetry after converting mirrored joints to a common sign convention.

Do not compare raw values like:

L_knee = +1.6

R_knee = -1.5

unless the joint convention has already been normalized.

Create a function like:

normalized_right_joint = sign_map[joint_name] * raw_right_joint

symmetry_error = left_joint - normalized_right_joint

Track rolling averages over **5 full gait cycles**:

hip_pitch_symmetry_error

hip_roll_symmetry_error

hip_yaw_symmetry_error

knee_symmetry_error

ankle_symmetry_error

step_length_symmetry_error

step_duration_symmetry_error

stance_duration_symmetry_error

full_sole_support_symmetry_error

contact_force_symmetry_error

Reject if persistent L/R asymmetry remains after sign normalization.

5. Step-quality metrics

For each L and R step:

heel_strike_time

toe_off_time

step_duration

step_length

step_width

swing_duration

stance_duration

double_support_duration
single_support_duration
full_sole_support_duration

Derived metrics:

cadence
stride_length
duty_factor_L/R
double_support_ratio
single_support_ratio_L/R
airborne_ratio
 $\text{step_length_per_velocity} = \text{step_length} / \text{commanded_vx}$
 $\text{expected_step_length} = \text{measured_vx} * \text{step_duration}$

Reject or penalize if:

step length is too short
step duration is irregular
left/right step length differs too much
double support is abnormal
airborne time exists during walking
cadence is high but stride is tiny

The rolling average should be **5 full gait cycles**, not a fixed 3-second window.

6. Crouch / knee posture gates

Track:

root_height_mean
root_height_p5
knee_angle_mean_L/R
knee_angle_min_L/R
knee_angle_max_L/R
knee_range_L/R
ankle_angle_mean_L/R
hip_pitch_mean_L/R

Reject if:

root height is too low
knee mean indicates crouch-walking
knee range is too small, meaning locked crouch
hip/ankle compensation is extreme

A policy should not be accepted just because it walks forward while sitting into the knees.

7. Speed tracking after gait quality passes

Only compare speed once the checkpoint passes contact, roll, symmetry, and posture gates.

Metrics:

vx_mean
 vx_error_mean
 vx_error_p95
 $vx_tracking_ratio = vx_mean / cmd_vx$

For example:

$cmd = 0.52 \text{ m/s}$
 $vx = 0.45 \text{ m/s}$
tracking ratio = 86.5%

This is usable during development, but a better checkpoint should improve it without worsening tiptoe, crouch, roll bias, or symmetry.

Updated approve/reject rule

Use this logic:

A checkpoint is APPROVED only if:

1. It survives and walks without body contact.
2. It tracks yaw and lateral direction acceptably.
3. It does not tiptoe.
4. It has real full-sole support during stance.
5. Its roll oscillation is centered near neutral, with low persistent roll mean.

6. Left/right joint averages are symmetric after sign normalization.
7. Knees are not excessively bent.
8. Step length and cadence are physically reasonable.
9. It tracks commanded vx reasonably.
10. Video agrees with the metrics.

Reject examples:

REJECT: walks straight but tiptoes.
REJECT: yaw is good but persistent left roll bias remains.
REJECT: speed improved but knees are more crouched.
REJECT: reward improved but step length collapsed.
REJECT: foot-flat reward improved but heel contact is absent.
REJECT: average torso roll is nonzero even though roll RMS is small.

Important dashboard change

The dashboard should show **bias and oscillation separately**:

roll_mean -> persistent lean
roll_rms_centered -> oscillation around the lean
raw_roll_rms -> total roll motion, less useful alone

Same for:

torso roll
box/top roll
hip roll
hip yaw
lateral COM offset

This matters because the current robot may have unavoidable roll oscillation, but the unacceptable part is that the oscillation is centered around a tilted left posture.

Minimal dashboard panels

For every checkpoint, show:

1. Survival / termination causes
2. vx, vy, yaw, heading drift
3. torso/box roll mean vs centered RMS
4. hip roll/yaw mean vs centered RMS
5. L/R normalized joint rolling averages
6. foot contact timeline: heel, toe, full sole
7. step events and gait cycles
8. step length / step duration / cadence
9. double support / single support / full sole support ratios
10. root height and knee angle distributions
11. action rate / torque / joint velocity
12. old checkpoint vs new checkpoint scorecard

The key line for Codex is:

Do not let a policy pass just because yaw is fixed and reward is high. The current failure is stable-but-bad walking: tiptoe contact, persistent roll bias, asymmetric joint averages, crouched knees, short steps, and under-speed.